

Movie Recommendation System

Using Heap-Based Collaborative Filtering

Data Structures — Project-2 Report

Roaa Shalab_Alsham – 2421051350

Süheyla Yusuf - 2321051371

1. Introduction

This report documents the design, implementation, and testing of a Movie Recommendation System built in Java as the second programming project for the Data Structures course. The system applies heap-based collaborative filtering to find users with similar taste to a given target and recommends movies they enjoyed that the target has not yet rated.

Collaborative filtering is one of the most widely used techniques in modern recommender systems (Netflix, Spotify, Amazon). The “heap-based” variant specifically uses a max-heap to efficiently retrieve the top-K most similar users without having to fully sort all 600 users every time. This makes the approach both algorithmically correct and pedagogically meaningful for demonstrating the heap data structure.

1.1 Project Goals

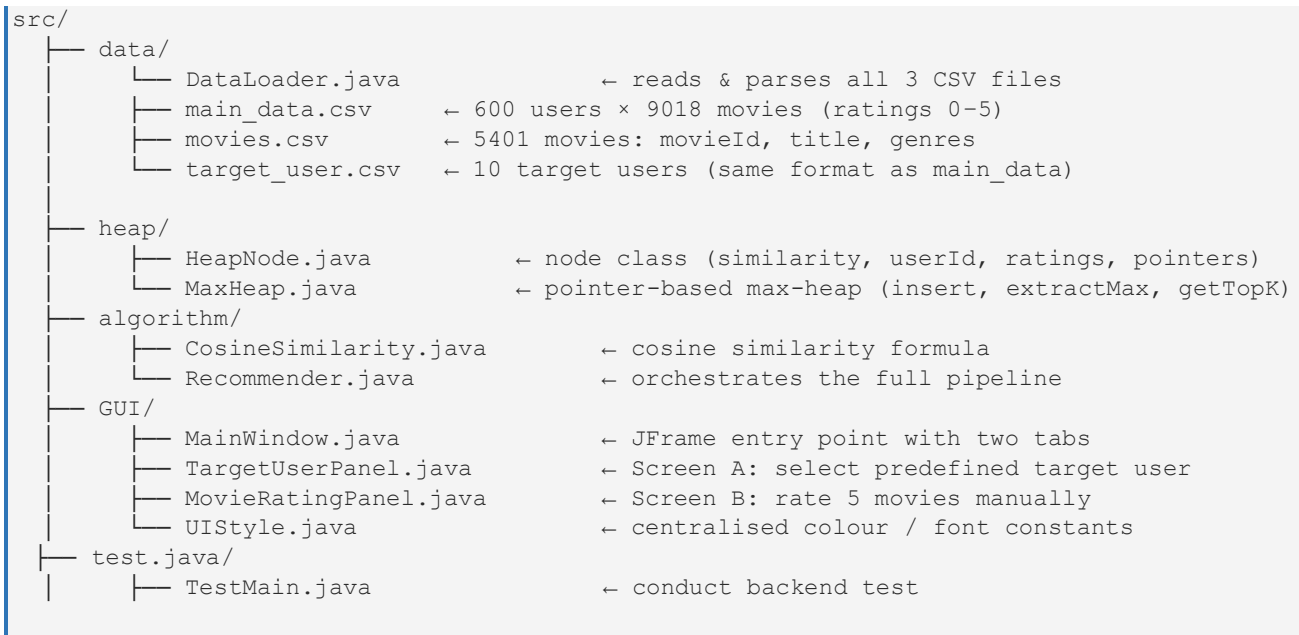
- Load a 600-user × 9018-movie rating matrix from CSV files.
- Compute cosine similarity between a target user and every other user.
- Store users in a pointer-based max-heap ordered by similarity.
- Extract the top-X most similar users and retrieve their top-K movies.
- Present the results through a Java Swing GUI with two separate screens.

1.2 Technology Stack

Component	Technology / Details
Language	Java (JDK 17+)
IDE	Apache NetBeans (project structure included)
GUI Framework	Java Swing with Nimbus Look and Feel
Build System	Apache Ant (build.xml provided)
Data format	CSV (comma-separated values)

2. System Architecture

The codebase is organised into four packages, each with a single clearly defined responsibility. This separation ensures that data loading, core algorithm logic, the heap data structure, and the GUI are all independently testable and replaceable.



2.1 Data Flow Diagram



3. Data Files & Chosen Data Structures

3.1 Input Files

main_data.csv

600 rows × 9019 columns. The first column is user_id; the remaining 9018 columns are movie IDs used as column headers. Each cell value is an integer rating 0–5, where 0 means the movie was not rated.

```
user_id, 1, 2, 3, 4, 5, ..., 9018
1, 4, 0, 0, 0, 4, ..., 0
2, 0, 3, 0, 5, 0, ..., 2
...
```

movies.csv

5401 rows. Columns: movieId, title, genres. Some titles contain commas (e.g. “Grumpier Old Men, The”), so parsing must use a split limit of 3 rather than splitting on every comma.

```
movieId, title, genres
```

1, Toy Story (1995),	Adventure Animation Children Comedy Fantasy
2, Jumanji (1995),	Adventure Children Fantasy
3, Grumpier Old Men (1995),	Comedy Romance

target_user.csv

10 rows (user IDs 601–610). Identical format to main_data.csv. These rows serve as the “new” users for whom recommendations are generated. They appear in the Screen A combo box.

3.2 Chosen Data Structures & Justifications

User-Movie Matrix — LinkedHashMap<Integer, float[]>

Chosen: LinkedHashMap<Integer, float[]>
 Key = userId (Integer) | Value = ratings array (float[], length 9018)

The system needs two operations on this structure: iterating over every user to compute similarities, and looking up a specific user by ID in $O(1)$. The choice was made against these alternatives:

Structure	$O(\text{lookup by userId})$	Preserves CSV order
2D int[][]	$O(1)$ but needs manual index mapping	Yes
ArrayList<float[]>	$O(n)$ linear scan	Yes
HashMap<Integer, float[]>	$O(1)$	No — arbitrary
LinkedHashMap (chosen)	$O(1)$	Yes — insertion order

Why **float[]** and not **double[]** or **int[]**? Ratings are integers 0–5 which fit in a float with zero precision loss. float uses 4 bytes vs double’s 8 bytes. With 600 users $\times$ 9018 movies, float[] costs ~21 MB whereas double[] would cost ~43 MB for no benefit.

Movie Title Lookup — HashMap<Integer, String>

movies.csv maps movieId $\rightarrow$ title. A plain HashMap gives $O(1)$ lookup per movieId. This lookup happens for every candidate movie for every similar user during recommendation generation, so $O(1)$ per call is important. No ordering is needed, so LinkedHashMap is not required here.

Column Index Reverse Map — int[] colIndexToMovieId

DataLoader parses the CSV header and builds movieId $\rightarrow$ colIndex (a HashMap). The Recommender then inverts this into a primitive int[] array indexed by column position, so that a column index can be converted back to a movieId in $O(1)$ without a HashMap.get() call inside the inner recommendation loop.

Pointer-Based Max-Heap — HeapNode + MaxHeap

A max-heap is a complete binary tree where every parent’s value $\geq$ its children’s values, so the root always holds the maximum. After all 600 users are inserted ordered by their cosine similarity, the root is the most similar user. Calling `extractMax()` X times retrieves the top-X users in $O(X \log n)$.

4. Heap Implementation

4.1 HeapNode

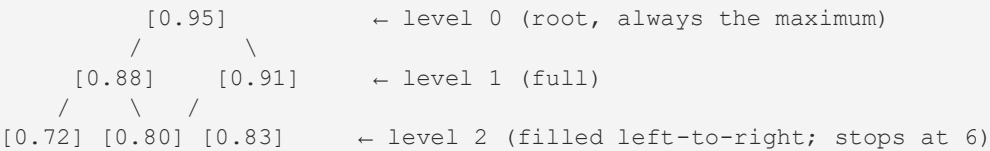
Each node in the heap is an instance of `HeapNode`. It holds three data fields and three structural pointer fields:

Field	Purpose
double similarity	The cosine similarity score — what the heap is ordered by (max at root).
int userId	The row ID from <code>main_data.csv</code> that this node represents.
float[] ratings	The user’s full 9018-element ratings vector. Stored directly on the node so that once a node is extracted, its movie data is immediately available without a second <code>HashMap</code> lookup.
HeapNode left	Left child pointer (null if no left child).
HeapNode right	Right child pointer (null if no right child).
HeapNode parent	Parent pointer (null for the root). Required by <code>heapify-up</code> so the algorithm can walk upward after a bottom insertion.

4.2 The Complete Binary Tree Shape Rule

A heap must always be a COMPLETE binary tree: every level is fully filled except the last, and the last level is filled left-to-right. This shape rule is what allows $O(\log n)$ guarantees because the tree height is always $\lceil \log_2(n) \rceil$.

Example: 6 nodes in a complete binary tree



Node 7 (next insert) would go as the right child of [0.91].

4.3 The Binary Path Trick — Navigating Without an Array

In a traditional array-based heap, finding any node by its 1-based index is trivial (just access the array at that index). In a pointer-based heap, we must navigate the tree. The key insight is that the binary representation of a node’s 1-based index encodes the exact path from root to that node:

Algorithm: `navigateToPosition(index)`

1. Convert index to binary: e.g. `index=6` → binary `110`
2. Remove the leading 1-bit: → `10`
3. Follow remaining bits from root:

```
0 = go left
1 = go right
4. Result: you arrive at node 6
```

More examples:

```
index 4 → binary 100 → skip 1 → 00 → left, left
index 5 → binary 101 → skip 1 → 01 → left, right
index 7 → binary 111 → skip 1 → 11 → right, right
```

This function is used in two places: (1) to find the PARENT of the next insertion slot (navigate to $\text{size}/2$), and (2) to find the LAST node during `extractMax` (navigate to `size`).

4.4 Insert + Heapify-Up

Insertion maintains the complete-tree shape by always placing the new node at the next available left-to-right position on the bottom level.

1. Increment size. The new node will occupy position `size` in the 1-based numbering.
2. Navigate to position $\text{size}/2$ — this is the parent of the new node.
3. If `size` is even, attach as the LEFT child; if odd, attach as the RIGHT child.
4. Set the new node's parent pointer to the found parent.
5. Call `heapifyUp`: walk upward swapping DATA (not pointers) while the node's similarity exceeds its parent's.

Heapify-Up example: insert `similarity=0.95` into:



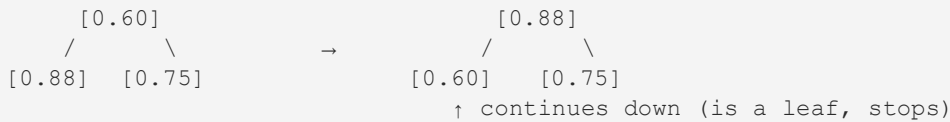
```
Step 1: 0.95 > 0.90 (parent) → swap data
Step 2: now at root, no parent → stop
```

4.5 ExtractMax + Heapify-Down

To remove the maximum (root), we cannot simply delete it — that would break the tree structure. Instead we copy the LAST node's data into the root and delete the last node (which is always a leaf), then push the root value downward.

6. Save root's data to return as the result.
7. Navigate to position `size` (the last node). Copy its data into root.
8. Detach the last node: set its parent's left or right pointer to null, then set `lastNode.parent = null`. (Setting parent to null prevents stale pointer references. The node is then unreachable and garbage-collected.)
9. Decrement `size`.
10. Call `heapifyDown` from root: at each step, find the larger child; if it's bigger than the current node, swap data and continue downward. Stop when both children are smaller or the node is a leaf.

Heapify-Down example: after extractMax, last node (0.60) moved to root:



Why swap DATA instead of relinking pointers?

Swapping pointer references in a doubly-linked tree with parent pointers requires updating 6+ references (both children, both parents, sibling links, possibly root).

It is error-prone and adds no logical benefit.

Swapping only the three payload fields (similarity, userId, ratings) achieves exactly the same heap effect in 5 lines with no edge cases.

4.6 Time Complexity Summary

Operation	Time Complexity
insert(HeapNode)	$O(\log n)$ — tree height is $\lceil \log_2 n \rceil$; heapify-up does at most that many swaps
extractMax()	$O(\log n)$ — heapify-down does at most $\lceil \log_2 n \rceil$ swaps
getTopK(k)	$O(k \log n)$ — k calls to extractMax
navigateToPosition(i)	$O(\log n)$ — follows at most $\lceil \log_2 i \rceil$ bits
Full heap build (600 users)	$O(n \log n)$ where $n=600 \rightarrow \sim 5400$ comparisons total
Full pipeline	$O(n \cdot m + n \log n + k \log n)$ where $n=600$ users, $m=9018$ movies

5. Cosine Similarity

5.1 Why Cosine Similarity?

User rating vectors are highly sparse: most of the 9018 values are 0 (not rated). Two users may have both rated only 20 movies, with maybe 5 in common. Cosine similarity measures only the ANGLE between vectors, so it captures whether two users rate the same movies similarly regardless of how many movies each has rated.

5.2 Formula

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

$$A \cdot B = \sum (a_i \times b_i) \quad \text{dot product over all 9018 movie positions}$$

$||A|| = \sqrt{\sum a_i^2}$ Euclidean magnitude of vector A
 $||B|| = \sqrt{\sum b_i^2}$ Euclidean magnitude of vector B

Result: 0.0 (no overlap) to 1.0 (identical rating patterns)
Edge case: if either magnitude is 0 (all-zero vector) → return 0.0

5.3 Worked Example

User A = [5, 3, 0, 0, 4] — User B = [4, 0, 0, 0, 5]

```
Dot product = 5×4 + 3×0 + 0×0 + 0×0 + 4×5 = 20 + 0 + 0 + 0 + 20 = 40
||A||       = √(25 + 9 + 0 + 0 + 16) = √50 ≈ 7.071
||B||       = √(16 + 0 + 0 + 0 + 25) = √41 ≈ 6.403
cos(A, B)   = 40 / (7.071 × 6.403) ≈ 40 / 45.27 ≈ 0.883
```

Interpretation: 0.883 is quite close to 1.0 → users have very similar taste

5.4 Implementation Notes

The implementation accumulates dot product and both squared magnitudes in a single $O(m)$ loop to avoid iterating three times. Intermediate values are cast to double even though input is `float[]` to avoid floating-point precision loss during accumulation across 9018 elements.

6. Recommendation Pipeline

6.1 Full Algorithm — `getRecommendations(targetVector, X, K)`

```
Input: targetVector — float[9018] ratings for the target user
       X            — number of most similar users to retrieve
       K            — number of top movies to take from each similar user
Output: List<String> — X × K movie title strings
```

Step 1 — Similarity + Heap Build:

```
For each of 600 users in userMatrix:
    sim = CosineSimilarity.compute(targetVector, userVector) // O(9018)
    heap.insert(new HeapNode(userId, sim, ratings))           // O(log 600)
After all inserts: heap.root = most similar user
```

Step 2 — Extract Top-X:

```
topUsers = heap.getTopK(X) // X × extractMax, each O(log n)
topUsers[0] = most similar, topUsers[1] = second most similar, ...
```

```

Step 3 – Get Top-K Movies Per User:
For each similarUser in topUsers:
    For each column col in 0..9017:
        if similarUser.ratings[col] > 0 AND targetVector[col] == 0:
            collect (rating, colIndex)    // exclude already-rated movies
    Sort collected pairs by rating descending
    Take first K titles from movieTitles map
    Add to result list

Return: combined list of X×K movie titles

```

6.2 Movie Retrieval Detail

Inside `getTopKMoviesForUser()`, the similar user’s rated-but-not-seen-by-target movies are collected into a `List<float[]>` where each entry is `{rating, colIndex}`. This list is sorted by rating descending using Java’s `Arrays.sort` with a lambda comparator, and the first K entries are translated to movie titles via the `colIndexToMovieId` array and `movieTitles` `HashMap`. Simple sort is used here (not another heap) because each user has at most ~200 rated movies, making a sort perfectly fast ($200 \times \log_2 200 \approx 1500$ comparisons).

7. Data Loading — DataLoader

7.1 loadUserMatrix()

The method reads `main_data.csv` line by line using `BufferedReader` for memory efficiency. The header row is parsed first to build the `movieIdToColIndex` map (`movieId` → 0-based column position). Each subsequent row is split on commas to produce the `userId` and its 9018-element `float[]` ratings array, which is stored in the `LinkedHashMap`.

7.2 loadMovieTitles()

`movies.csv` is read with `split(",", 3)` — the limit-3 split is essential because some titles contain commas (e.g. “Grumpier Old Men, The (1995)”). Without the limit, such a title would be split into four tokens instead of three, corrupting the title field. The genre column (index 2) is ignored entirely.

7.3 loadTargetUsers()

`target_user.csv` has an identical format to `main_data.csv`, so `loadTargetUsers()` simply delegates to `loadUserMatrix()` with the target file path. The returned `LinkedHashMap` contains 10 entries (user IDs 601–610) whose keys populate the Screen A combo box and whose values are passed directly to `getRecommendations()`.

8. GUI Design

8.1 MainWindow

`MainWindow` extends `JFrame` and serves as the application entry point. Its constructor instantiates the `Recommender` (loading all three CSV files), then creates a `JTabbedPane` with two tabs: “Target User”

(TargetUserPanel) and “Movie Ratings” (MovieRatingPanel). The Nimbus look and feel is applied for a modern appearance. Window size is set to 900 × 600.

8.2 Screen A — TargetUserPanel

Screen A allows the user to select one of the 10 predefined target users and get movie recommendations. The panel layout (built with NetBeans GroupLayout) contains:

- JComboBox (userComboBox) — populated from recommender.getTargetUserIds() on construction. Shows user IDs 601–610.
- JTextField xField — user enters X (number of similar users to retrieve).
- JTextField kField — user enters K (top movies per similar user).
- JButton recommendButton — triggers getRecommendations() on click.
- JTextArea resultArea inside a JScrollPane — displays one movie title per line.

On button click, the handler reads the selected userId, fetches its vector, parses X and K, calls recommender.getRecommendations(), and appends each returned title to the text area on a new line. Any exception (invalid integer, empty field) is shown via JOptionPane.

8.3 Screen B — MovieRatingPanel

Screen B allows the user to manually rate 5 movies and receive recommendations based on those ratings. On construction, recommender.getRandomMovieTitles(100) fetches 100 random movie titles which are loaded into all five JComboBoxes.

The 5-row rating grid:

- movieCombo1–movieCombo5 — five JComboBoxes, each containing the same 100 random movie titles.
- ratingField1–ratingField5 — five JTextFields for entering integer ratings 1–5.

On button click, addMovieRating() is called for each row. It reads the selected title, looks up its column index via recommender.getColIndexForTitle(), and sets the corresponding position in the user vector float[]. The completed vector is passed to getRecommendations().

8.4 UIStyle

UIStyle is a utility class holding all colour and font constants. Centralising these ensures visual consistency across both panels without duplicating style code. The theme uses a dark background (#1E1E2E) with panel surfaces at (#313244) and an accent colour (#CBA6F7) for buttons, giving a modern dark-mode appearance.

Constant	Value
BACKGROUND	RGB(30, 30, 46) — deep navy dark background
PANEL	RGB(49, 50, 68) — slightly lighter surface
BUTTON	RGB(203, 166, 247) — lavender accent
TEXT	Color.WHITE
NORMAL_FONT	Segoe UI, plain, 18pt
BUTTON_FONT	Segoe UI, bold, 18pt

9. Class-by-Class Method Reference

HeapNode (heap package)

Method / Field	Description
HeapNode(userId, similarity, ratings)	Constructor. All three pointer fields (left, right, parent) initialised to null.
toString()	Returns "HeapNode{userId=N, similarity=X.XXXX}" for debugging.

MaxHeap (heap package)

Method	Description
insert(HeapNode)	Finds insertion parent via navigateToPosition(size/2), attaches node, heapifyUp. $O(\log n)$.
extractMax()	Returns root data, replaces with last node, removes last node, heapifyDown. $O(\log n)$.
getTopK(int k)	Calls extractMax() k times; returns list in descending similarity order. $O(k \log n)$.
size()	Returns current number of nodes.
isEmpty()	Returns true if size == 0.
navigateToPosition(index) [private]	Binary path traversal to find node at 1-based position. $O(\log n)$.
heapifyUp(node) [private]	Bubbles node upward swapping data while node > parent. $O(\log n)$.
heapifyDown(node) [private]	Pushes node downward swapping with larger child. $O(\log n)$.
swapData(a, b) [private]	Swaps similarity, userId, ratings between two nodes. $O(1)$.
printHeap() [debug]	BFS level-by-level print of the full tree. $O(n)$.

CosineSimilarity (algorithm package)

Method	Description
compute(float[] a, float[] b)	Single-pass $O(m)$ cosine similarity. Validates non-null and equal length. Returns 0.0 for zero vectors.

DataLoader (data package)

Method	Description
loadUserMatrix(path, movieIdToColIndex)	Reads main_data.csv or target_user.csv. Fills movieIdToColIndex as output parameter. Returns LinkedHashMap<userId, float[]>.

loadMovieTitles(path)	Reads movies.csv with split limit 3. Returns HashMap<movieId, title>.
loadTargetUsers(path, movieIdToColIndex)	Delegates to loadUserMatrix. Returns 10 target user entries.

Recommender (algorithm package)

Method	Description
Recommender(mainPath, moviesPath, targetPath)	Constructor: loads all CSVs, builds colIndexToMovieId reverse map.
getRecommendations(float[], X, K)	Main API: builds heap, extracts top-X, returns X×K movie titles.
getTargetUserIds()	Returns ordered list of target user IDs for combo box.
getTargetUserVector(userId)	Returns float[] for a given target userId.
getRandomMovieTitles(count)	Returns `count` shuffled movie titles for Screen B combo boxes.
getColIndexForTitle(title)	Translates movie title → ratings array column index.
createEmptyUserVector()	Returns float[numMovies] of zeros for Screen B to populate.
getTopKMoviesForUser(ratings, target, K) [private]	Collects, sorts, and returns top-K movie titles for one similar user.

10. Sample Output

10.1 Console Output (CLI Test Run)

Output - Movie-Recomindation-System-using-Heaps (run)

```
run:
[DataLoader] Loaded 600 users with 9018 movie columns from: src/data/main_data.csv
[DataLoader] Loaded 5401 movie titles from: src/data/movies.csv
[DataLoader] Loaded 10 users with 9018 movie columns from: src/data/target_user.csv
[Recommender] Ready. 600 main users, 10 target users, 5401 movie titles.
[Recommender] Heap built with 600 users.
[Recommender] Top 2 users extracted:
    userId=335    similarity=0.0749
    userId=103    similarity=0.0721
[Recommender] userId=335 contributed 1 movies.
[Recommender] userId=103 contributed 2 movies.
[Recommender] Heap built with 600 users.
[Recommender] Top 2 users extracted:
    userId=55     similarity=0.1442
    userId=287    similarity=0.1442
[Recommender] userId=55 contributed 0 movies.
[Recommender] userId=287 contributed 0 movies.
[Recommender] Heap built with 600 users.
[Recommender] Top 2 users extracted:
    userId=296    similarity=0.6088
    userId=593    similarity=0.5784
[Recommender] userId=296 contributed 3 movies.
[Recommender] userId=593 contributed 3 movies.
```

10.2 Screen A — Target User Recommendations

Movie Recommendation System

Target User

Movie Ratings

Target User

608

X:

2

K:

3

Recommend

Sense and Sensibility (1995)

"Cry

Dead Presidents (1995)

Sabrina (1995)

"American President

Dead Presidents (1995)

10.3 Screen B — Manual Movie Rating Recommendations

Movie Recommendation System

Target User

Movie Ratings

Neighbors (1981)

1

X: 4

Big Momma's House (2000)

4

K: 5

"Man Who Would Be King

2

Get Recommendations

Dragon Seed (1944)

5

Trust (1990)

3

Dangerous Minds (1995)

"Cry

Restoration (1995)

How to Make an American Quilt (1995)

Home for the Holidays (1995)

"Air Up There

Fearless (1993)

Up Close and Personal (1996)

Once Were Warriors (1994)

"Specialist

11. Conclusion

This project successfully implements a working movie recommendation system that demonstrates collaborative filtering using a custom pointer-based max-heap. The key data structure achievements are:

- A HeapNode class with left, right, and parent pointer fields that forms a true pointer-based binary tree — satisfying the project's requirement of not using arrays inside the heap.
- A MaxHeap that uses the binary path trick to navigate the complete binary tree by index without an array, enabling correct insert and extractMax operations in $O(\log n)$.
- A LinkedHashMap as the user-matrix container, balancing $O(1)$ userId lookup with predictable iteration order and memory efficiency via float[] over double[].
- A CosineSimilarity implementation that handles sparse vectors correctly, avoids division by zero, and accumulates in double precision for numerical stability.

The Swing GUI provides both required screens: target-user-based recommendations via a combo box, and custom-rating-based recommendations via a 5-row movie grid. The dark-themed UIStyle class centralises all visual constants to maintain consistent appearance across both panels.

Total lines of Java code: ~900

Classes: 9 (HeapNode, MaxHeap, CosineSimilarity, Recommender,
DataLoader, MainWindow, TargetUserPanel, MovieRatingPanel, UIStyle)

Dataset: 600 users × 9018 movies × 10 target users × 5401 movie titles